

United International University

CSE 3422: Software Engineering Lab

CRUD Operations

05/03/26

Contents

1 Learning Objectives **1**

2 What is CRUD? **1**

3 Technologies Used **1**

4 Step 1: Install Python **1**

5 Step 2: Install Flask **2**

6 Step 3: Create Project Folder **2**

7 Step 4: Creating the Database **2**

8 Step 5: Create the Flask Application **3**

9 Step 6: Helper Function for Database **3**

10 Step 7: Read Operation (Display Students) **3**

11 Step 8: Create Operation (Add Student) **4**

12 Step 9: Update Operation **4**

13 Step 10: Delete Operation **5**

14 Step 11: HTML Templates **6**

14.1 index.html 6

14.2 add.html 7

14.3 edit.html 7

15 Step 12: Run the Application **8**

16 Conclusion **8**

1 Learning Objectives

By the end of this lab, students will be able to:

- Understand the concept of CRUD operations
- Build a basic web application using Flask
- Store and retrieve data using SQLite
- Create simple HTML forms
- Connect a backend server with a database

2 What is CRUD?

CRUD represents the four basic operations that can be performed on data in a database.

- **Create** – Add new data
- **Read** – View existing data
- **Update** – Modify existing data
- **Delete** – Remove data

In this lab we will build a small application called:

Student Manager

This application will allow users to:

- Add a student
- View the list of students
- Edit a student's information
- Delete a student

3 Technologies Used

- Python
- Flask (Web Framework)
- SQLite (Database)
- HTML (User Interface)

4 Step 1: Install Python

Make sure Python is installed.

Check using:

```
python --version
```

If Python is not installed, download it from:

`https://python.org`

5 Step 2: Install Flask

Install Flask using pip:

```
pip install flask
```

6 Step 3: Create Project Folder

Create a new folder:

```
student_crud_app
```

Inside this folder create:

- `app.py`
- a folder called `templates`

Your project structure should look like this:

```
student_crud_app
|
|-- app.py
|
|-- templates
    |-- index.html
    |-- add.html
    |-- edit.html
```

7 Step 4: Creating the Database

Create a new file called:

`init_db.py`

```
import sqlite3

conn = sqlite3.connect("students.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
```

```
    department TEXT NOT NULL
)
""")

conn.commit()
conn.close()

print("Database created successfully!")
```

Run the file:

```
python init_db.py
```

This will create the SQLite database.

8 Step 5: Create the Flask Application

Now open `app.py`

```
from flask import Flask, render_template, request, redirect
import sqlite3

app = Flask(__name__)
```

9 Step 6: Helper Function for Database

Add this function to connect to the database.

```
def get_db_connection():
    conn = sqlite3.connect("students.db")
    conn.row_factory = sqlite3.Row
    return conn
```

10 Step 7: Read Operation (Display Students)

Add this route to display all students.

```
@app.route("/")
def index():

    conn = get_db_connection()

    students = conn.execute(
```

```
        "SELECT * FROM students"
    ).fetchall()

    conn.close()

    return render_template(
        "index.html",
        students=students
    )
```

11 Step 8: Create Operation (Add Student)

Add the route for adding a new student.

```
@app.route("/add", methods=("GET", "POST"))
def add():

    if request.method == "POST":

        name = request.form["name"]
        email = request.form["email"]
        department = request.form["department"]

        conn = get_db_connection()

        conn.execute(
        "INSERT INTO students (name,email,department) VALUES
   (?,?,?)",
        (name,email,department)
        )

        conn.commit()
        conn.close()

        return redirect("/")

    return render_template("add.html")
```

12 Step 9: Update Operation

Add the update route.

```
@app.route("/edit/<int:id>", methods=("GET","POST"))
def edit(id):

```

```
    conn = get_db_connection()

    student = conn.execute(
        "SELECT * FROM students WHERE id = ?",
        (id,)
    ).fetchone()

    if request.method == "POST":

        name = request.form["name"]
        email = request.form["email"]
        department = request.form["department"]

        conn.execute(
        "UPDATE students SET name=?,email=?,department=? WHERE id
   =?",
        (name,email,department,id)
        )

        conn.commit()
        conn.close()

        return redirect("/")

    conn.close()

    return render_template(
        "edit.html",
        student=student
    )
```

13 Step 10: Delete Operation

Add the delete route.

```
@app.route("/delete/<int:id>")
def delete(id):

    conn = get_db_connection()

    conn.execute(
        "DELETE FROM students WHERE id=?",
        (id,)
    )

    conn.commit()
    conn.close()
```

```
13
14      return redirect("/")
```

Finally run the application:

```
if __name__ == "__main__":
    app.run(debug=True)
```

14 Step 11: HTML Templates

14.1 index.html

```
<h1>Student List</h1>

<a href="/add">Add Student</a>

<table border="1">

<tr>
<th>Name</th>
<th>Email</th>
<th>Department</th>
<th>Actions</th>
</tr>

{% for student in students %}

<tr>

<td>{{ student["name"] }}</td>
<td>{{ student["email"] }}</td>
<td>{{ student["department"] }}</td>

<td>
<a href="/edit/{{ student['id'] }}">Edit</a>
<a href="/delete/{{ student['id'] }}">Delete</a>
</td>

</tr>

{% endfor %}

</table>
```

14.2 add.html

```
<h1>Add Student</h1>

<form method="POST">

Name:
<input type="text" name="name">

<br>

Email:
<input type="text" name="email">

<br>

Department:
<input type="text" name="department">

<br>

<button type="submit">Add</button>

</form>
```

14.3 edit.html

```
<h1>Edit Student</h1>

<form method="POST">

Name:
<input type="text" name="name" value="{{ student['name'] }}">

<br>

Email:
<input type="text" name="email" value="{{ student['email'] }}">

<br>

Department:
<input type="text" name="department" value="{{ student['
    department'] }}">

<br>

```

```
<button type="submit">Update</button>

</form>
```

15 Step 12: Run the Application

Run the server:

```
python app.py
```

Open a browser and go to:

`http://127.0.0.1:5000`

You should now be able to:

- Add students
- View students
- Edit students
- Delete students

16 Conclusion

In this lab we built a complete CRUD application using:

- Flask as the backend server
- SQLite as the database
- HTML forms as the user interface

Students should now understand how web applications interact with databases using CRUD operations.